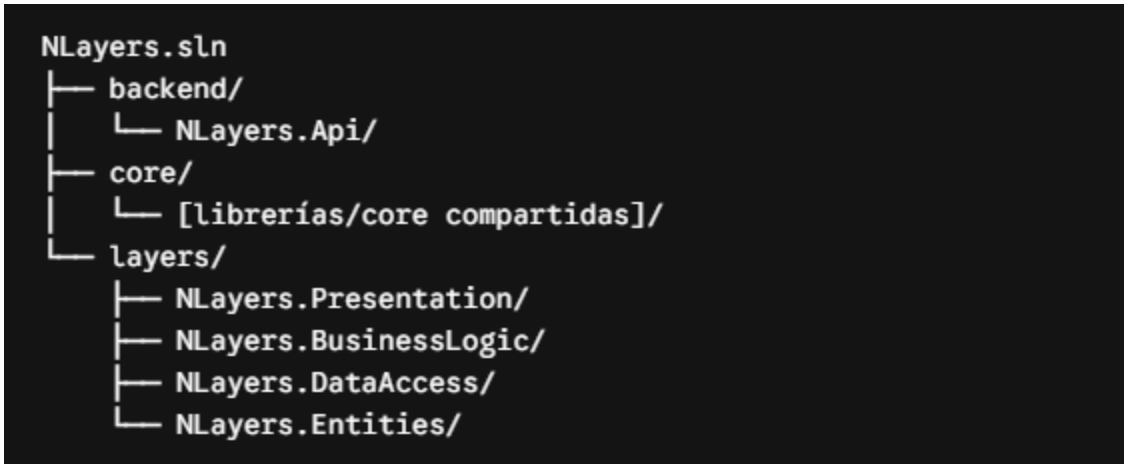
Guía para crear una API REST con arquitectura N-Layers en C#

Objetivo

El objetivo de este documento es proporcionar una guía como un procedimiento reproducible para desarrollar una API REST en **C# .NET**, aplicando una arquitectura basada en capas, que adicionalmente explique la responsabilidad que tiene cada capa, de quién depende y cómo fluye la información en relación a una petición HTTP hasta la base de datos, y de vuelta al cliente.

Estructura del proyecto



La solución estará organizada en diferentes proyectos, donde cada uno tendrá una responsabilidad específica.

Se trabajará con las siguientes capas:

- **Presentation:** Es la capa responsable de recibir las solicitudes del cliente, interpretar sus datos, invocar las operaciones de negocio correspondientes y construir la respuesta que será enviada al cliente.

- **BusinessLogic:** Es la capa responsable de aplicar las validaciones y ejecutar la lógica que está relacionada al negocio. Proporciona servicios para integraciones con sistemas externos como parte de los procesos de negocio.

- **DataAccess:** Es la capa responsable de generar las consultas hacia la base de datos, preparando la información que debe enviar a la misma.

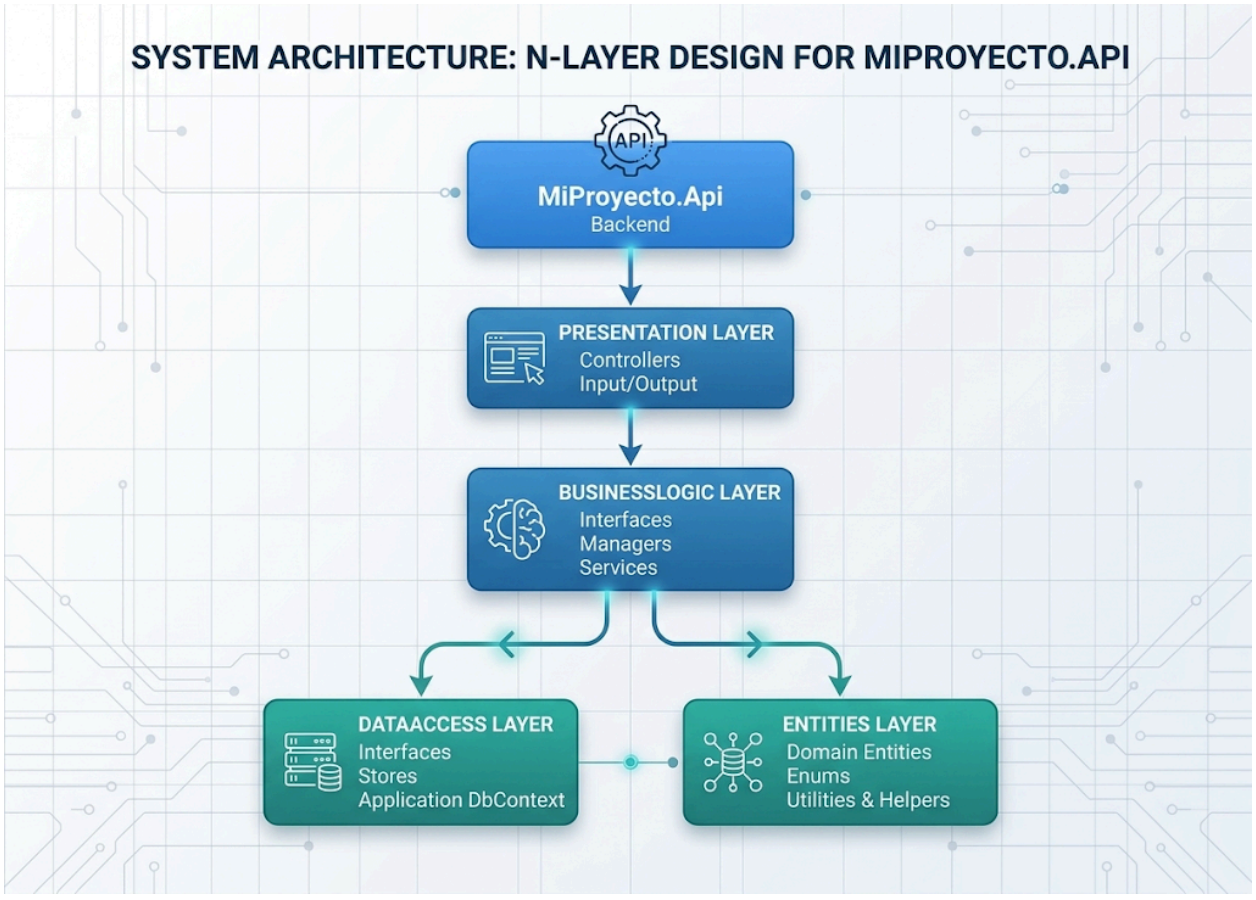
- **Entities:** Es la capa de soporte en donde se almacenan entidades y elementos compartidos como utilities o helpers.

Además, se utilizarán proyectos de soporte:

- **Backend:** Representa nuestra aplicación API Rest
- **Core:** Librerías con funcionalidades Core de las cuales depende nuestra arquitectura.

La principal finalidad de esta organización es **separar responsabilidades**, reducir el acoplamiento entre componentes y facilitar el mantenimiento y evolución de la aplicación.

Dependencias



Concepto de N-Layers

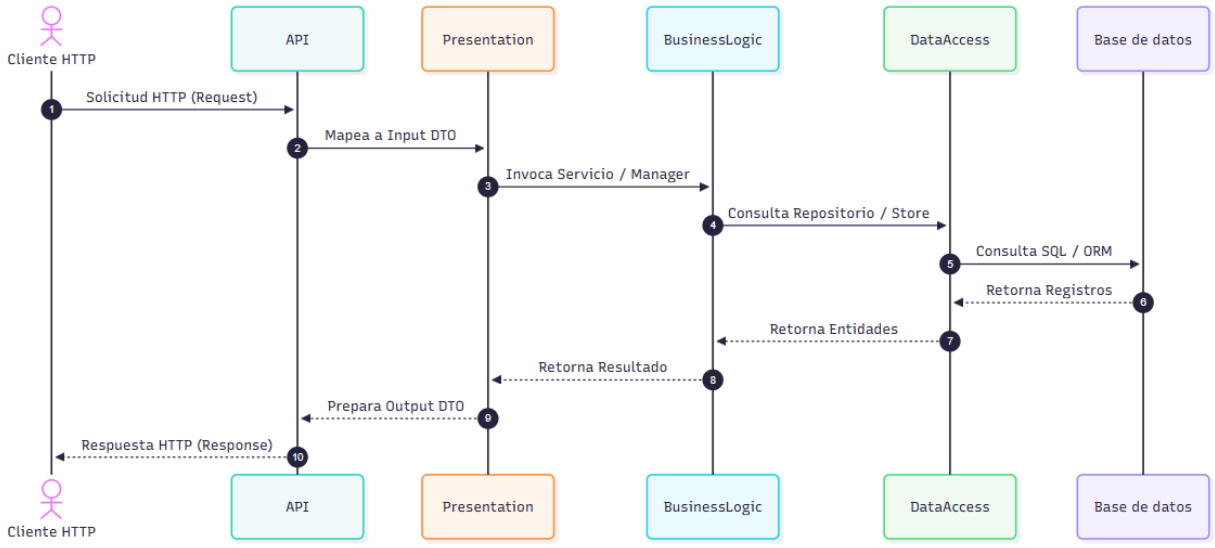
Una arquitectura en N-Layers divide una aplicación en diferentes capas lógicas.

Cada capa se ocupa de un tipo determinado de responsabilidad y se comunica con otras capas mediante contratos bien definidos.

En nuestra implementación utilizaremos cuatro capas principales:

Capa	**Responsabilidad**
Presentation	Recibir y responder solicitudes HTTP
BusinessLogic	Implementar las reglas de negocio
DataAccess	Acceder y persistir información
Entities	Representar objetos y elementos compartidos

Flujo de datos



Crear la solución

El primer paso será crear una **solución en blanco**.

Desde Visual Studio:

Create a new project → Blank Solution

Nombre:

NLayer.Architecture.Template.sln

La solución inicialmente no contendrá ningún proyecto.

Organización de la solución

Dentro de la solución se crearán tres carpetas:

NLayer.Architecture.Template.sln

```
├── backend
├── core
└── layers
```

Estas carpetas tienen como finalidad organizar físicamente los diferentes proyectos que componen la solución.

Backend

Contendrá el proyecto ejecutable de la API.

```
backend
└── NLayers.Api
```

Core

Contendrá librerías de infraestructura o funcionalidades compartidas por diferentes proyectos.

```
core
├── NLayers.BusinessLogic.Services.Http
└── NLayers.DataAccess.Stores.Sql
```

Layers

Contendrá los proyectos correspondientes a las capas de la arquitectura.

layers
├── NLayers.Presentation
├── NLayers.BusinessLogic
├── NLayers.DataAccess
└── NLayers.Entities

Crear el proyecto API

Dentro de la carpeta backend se agrega un proyecto: **ASP.NET Core Web API**

Por ejemplo:

NLayers.Api

Este será el **punto de entrada de la aplicación**.

Es importante comprender que la API no contendrá la lógica de negocio principal.

Su responsabilidad será principalmente:

- iniciar la aplicación;
- configurar servicios;
- configurar middleware;
- configurar autenticación/autorización;
- configurar Swagger;
- configurar dependencias;
- exponer los endpoints HTTP.

La API accede a la arquitectura a través de **Presentation**.

Crear las librerías de las capas

Dentro de layers se crearán cuatro proyectos de tipo: **Class Library (Biblioteca de Clases)**

Los proyectos serán:

- NLayers.Presentation
- NLayers.BusinessLogic
- NLayers.DataAccess
- NLayers.Entities

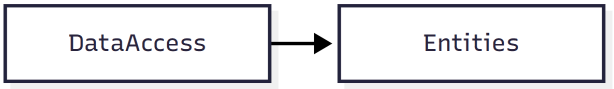
Configurar las dependencias (Paso crítico)

Este es uno de los puntos **más importantes de la práctica**. Los proyectos no deben referenciarse libremente. Se debe respetar la dirección de dependencias definida para la arquitectura.

Recordemos: Para poder referenciar un proyecto en otro, tenemos que hacer clic derecho sobre el proyecto → Agregar → Referencia de proyecto…

DataAccess → Entities

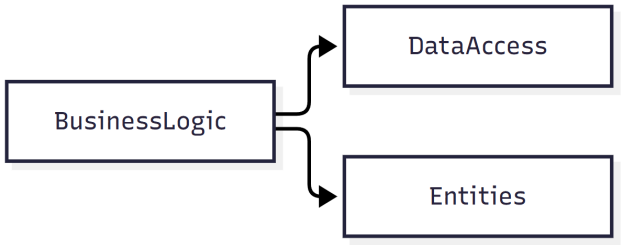
DataAccess necesita conocer las entidades con las cuales trabaja.



BusinessLogic → DataAccess + Entities

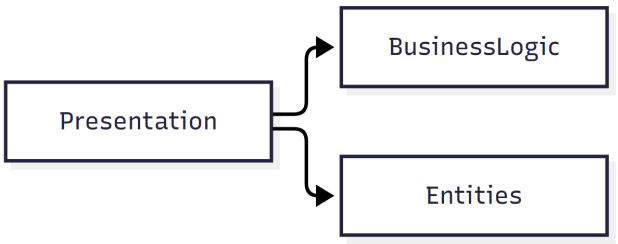
BusinessLogic utiliza:

- las interfaces y componentes de acceso a datos;
- las entidades de dominio.



Presentation → BusinessLogic + Entities

Presentation necesita acceder a BusinessLogic para ejecutar operaciones y a Entities para trabajar con determinados tipos compartidos.



API → Presentation

La API accederá a la arquitectura mediante Presentation.

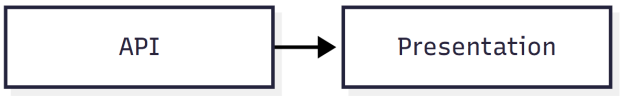
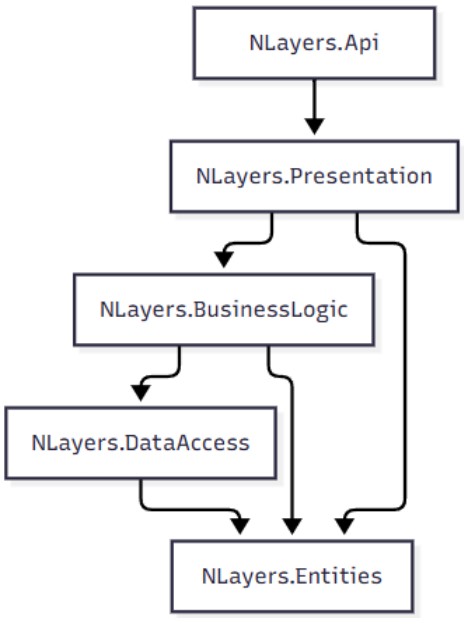


Diagrama completo de dependencias

La estructura final de referencias será:



Regla Fundamental: Una capa **no debería depender de una capa superior**.

DataAccess ❌ → BusinessLogic
BusinessLogic ❌ → Presentation
Entities ❌ → BusinessLogic
Entities ❌ → DataAccess

Esto evita generar dependencias circulares y mantiene separadas las responsabilidades.

Capa Entities

La capa Entities funciona como una **capa de soporte**. Su finalidad es contener objetos y elementos que serán utilizados por diferentes capas, pero en general no contiene más que definiciones de los modelos o entidades con las que trabaja la arquitectura. Esto hace que esta capa sea netamente reutilizable, permitiendo implementarla en diferentes aplicaciones.

Por ejemplo, si usted genera una clase especializada en hacer todo tipo de validaciones para enteros, strings, booleanos, esta clase contiene funcionalidad base que puede ser reutilizada en cualquier proyecto.

La estructura propuesta será:

NLayers.Entities
├── Entities
├── Enums
├── Utils
└── Helpers

Entities

Aquí se definirán las entidades que representan información de la aplicación. En general, para esta arquitectura las entidades representan objetos anémicos, es decir, que contienen propiedades o atributos, pero que carecen de lógica de negocio o comportamiento alguno.

Por ejemplo:

```
public class Product
{
   public int Id { get; set; }
   public string Name { get; set; }
   public decimal Price { get; set; }
}
```

La entidad representa un producto.

Posteriormente podrá ser utilizada por DataAccess para persistir información y por BusinessLogic para aplicar reglas de negocio.

Enums

Aquí se definirán enumeraciones.

Ejemplo:

```
public enum ProductStatus
{
   Active = 1,
   Inactive = 2
}
```

Utils

Los Utils son clases, definidas preferentemente por el modificador de acceso **static** y podrán contener utilidades generales que sean compartidas por diferentes componentes.

Por ejemplo:

- DateUtils: Utilidades para la manipulación y operaciones con Fechas.
- StringUtils: Utilidades para la manipulación, validaciones y operaciones con strings.
- ValidationUtils: Utilidades para validaciones en general.

La idea de los Utils, es evitar duplicar código común.

Helpers

Los Helpers son clases, también definidas preferentemente por el modificador de acceso **static**, podrán utilizarse para funcionalidades auxiliares. Cumplen más o menos la misma función que los utils, pero se los destina generalmente a manejar fragmentos de código para realizar tareas u operaciones comunes, por ejemplo el algoritmo de validación del dígito verificador de un CUIL, y su existencia también busca evitar duplicidad de código, facilitando el mantenimiento del sistema.

Por ejemplo:

- PasswordHelper
- TokenHelper
- MappingHelper

Dependiendo del proyecto, algunas de estas responsabilidades podrían posteriormente trasladarse a servicios específicos.

Capa DataAccess

La capa DataAccess o Acceso a Datos, tendrá como responsabilidad el acceso a los datos.La estructura propuesta será:

NLayers.DataAccess
├── Interfaces
├── Stores
└── AppDbContext.cs

AppDbContext

Esta será la clase encargada de representar la sesión con la base de datos mediante Entity Framework Core.

Ejemplo:

```
public class AppDbContext : DbContext
{
   public AppDbContext(DbContextOptions<AppDbContext> options)
      : base(options)
   {
   }

   public DbSet<Product> Products { get; set; }
}
```

El DbContext de EF permitirá realizar operaciones como:

- Consultar datos;
- Insertar registros;
- Modificar registros;
- Eliminar registros;
- Administrar relaciones;
- Ejecutar consultas mediante LINQ.

Interfaces de DataAccess

Las interfaces o contratos de esta capa, definirán las operaciones que estarán disponibles para acceder a los datos.

Por ejemplo:

```
public interface IProductStore
{
   Task<Product> GetByIdAsync(int id);
   Task<List<Product>> GetAllAsync();
   Task<Product> CreateAsync(Product product);
   Task UpdateAsync(Product product);
   Task DeleteAsync(int id);
}
```

Recordemos: La interfaz como contrato establece **qué operaciones existen**, pero no cómo se realizan (como abstracción). Esto nos da independencia separandonos de la implementación (Clase concreta y/o tecnología implementada).

Stores

Los Stores son clases que representan las implementaciones concretas de las interfaces de acceso a datos. Están relacionadas a una entidad específica (Un Store representa la manipulación de una tabla en DB, por ejemplo Product). Es por ello que en general, definiremos un Store por cada tabla que tengamos en DB.

Ejemplo:

```
public class ProductStore : IProductStore
{
   private readonly AppDbContext _context;

   public ProductStore(AppDbContext context)
   {
      _context = context;
   }

   public async Task<Product> GetByIdAsync(int id)
   {
      return await _context.Products.FirstOrDefaultAsync(x => x.Id == id);
   }

   public async Task<List<Product>> GetAllAsync()
   {
      return await _context.Products.ToListAsync();
   }
```

}

El Store conoce:

- Entity Framework Core;
- AppDbContext;
- tablas;
- consultas;
- persistencia.

Pero **no debería conocer HTTP ni Controllers**.

Capa BusinessLogic

La capa BusinessLogic o Lógica de Negocio contiene la lógica de negocio de la aplicación.

La estructura será:

NLayers.BusinessLogic
├── Interfaces
├── Managers
└── Services

Interfaces de BusinessLogic

Las interfaces o contratos de esta capa, definirán las operaciones de negocio disponibles.

Por ejemplo:

```
public interface IProductManager
{
   Task<Product> GetProductAsync(int id);
   Task<List<Product>> GetProductsAsync();
   Task<Product> CreateProductAsync(Product product);
   Task UpdateProductAsync(Product product);
   Task DeleteProductAsync(int id);
}
```

La interfaz representa el contrato que Presentation utilizará.

Managers

Los Managers son clases encargadas de coordinar las operaciones de negocio, y que están relacionadas a una entidad específica. Es por ello que tendremos que crear un Manager, para cada entidad que manipulamos en nuestra aplicación.

Por ejemplo:

```
public class ProductManager : IProductManager
{
   private readonly IProductStore _productStore;

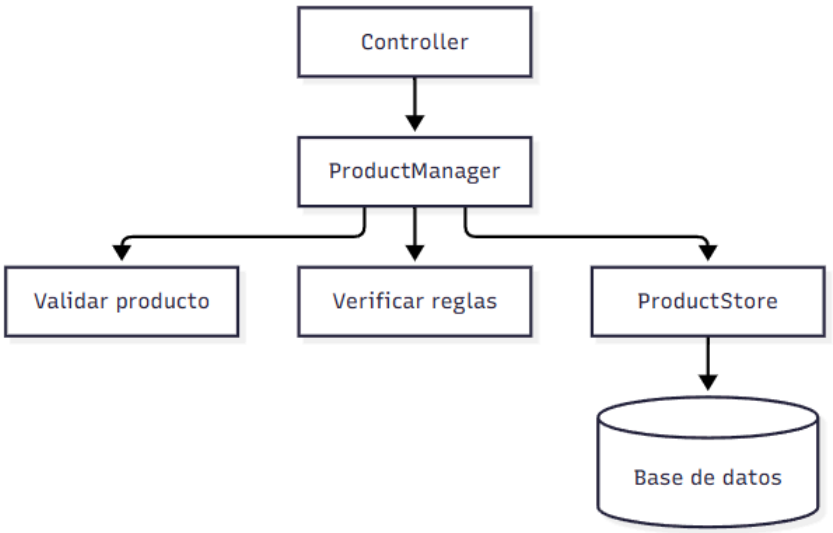
   public ProductManager(IProductStore productStore)
   {
      _productStore = productStore;
   }

   public async Task<Product> GetProductAsync(int id)
   {
      return await _productStore.GetByIdAsync(id);
   }
}
```

El Manager puede:

- validar reglas;
- verificar condiciones;
- coordinar varios Stores;
- llamar a Services;
- transformar información;
- controlar el flujo de una operación.

Por ejemplo:



Services

Los Services son clases que se especializan en una funcionalidad específica, por ejemplo enviar un email mediante Google. Esto hace que cada servicio encapsule internamente toda la lógica necesaria para el acceso a dicho servicio, incluyendo la manipulación de objetos de entrada y salidas relacionados al mismo.

Los Servicios pueden ser utilizados por los Managers u otros componentes de la lógica de negocio.

Por ejemplo:

- GoogleEmailService
- PasswordService
- MicrosoftTokenService
- WhatsappNotificationService

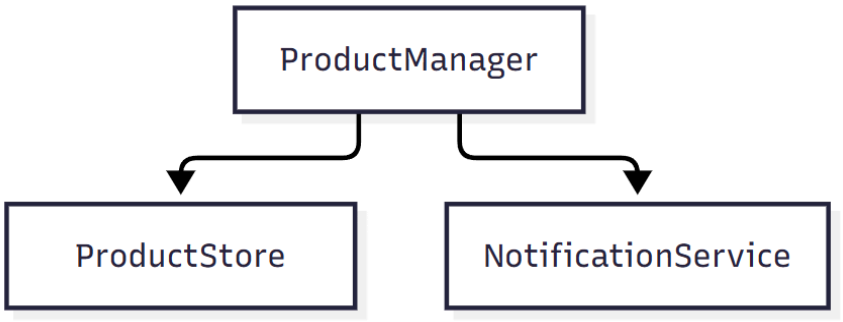
Un ejemplo:

```
public class EmailService
{
   public Task SendAsync(string email, string message)
   {
      // Implementación
   }
}
```

Diferencia conceptual

Manager: Coordina una operación de negocio.

Service: Proporciona una funcionalidad específica que puede ser utilizada durante esa operación.



Capa Presentation

La capa Presentation será la capa responsable de representar la interfaz HTTP de nuestra aplicación.

Su estructura puede ser:

NLayers.Presentation
├── Controllers
├── Base
├── Inputs
└── Outputs

Controllers

Los Controllers recibirán las solicitudes HTTP. Estos controladores serán controladores base para los controladores de la API.

Por ejemplo:

```
[ApiController]
[Route("api/products")]
public class ProductController : ControllerBase
```

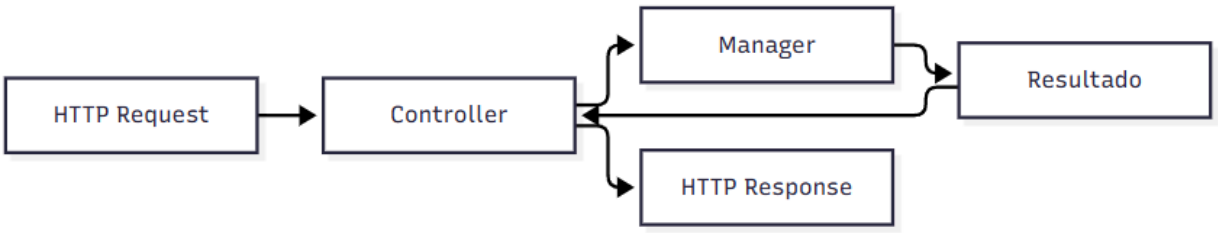
```
{
    private readonly IProductManager _productManager;

    public ProductController(IProductManager productManager)
    {
        _productManager = productManager;
    }

    [HttpGet("{id}")]
    public async Task<IActionResult> Get(int id)
    {
        var product = await _productManager.GetProductAsync(id);
        return Ok(product);
    }
}
```

El Controller debe ser lo más simple posible.

Su responsabilidad principal es:



No debería contener reglas de negocio complejas.

Controllers Base

Se puede crear un Controller base para centralizar funcionalidades comunes. Esto es para el caso por ejemplo en el que se quiera dar un formato Custom a las respuestas de nuestra api.

Por ejemplo, una manera típica de customizar las respuestas de nuestra api, es para que independientemente de si la respuesta es success o errónea, presente siempre la misma estructura:

```
{
    code: 200
    data: {“id”: 1234, “producto”: “coca-cola x 500 cc”}
    message: “El producto se creó exitosamente”
}
```

Por ejemplo:

```
public abstract class BaseController : ControllerBase
{
   protected IActionResult Success(object data)
   {
      return Ok(data);
   }
}
```

Los Controllers podrán heredar de él:

```
public class ProductController : BaseController
{
}
```

Esto permite centralizar comportamientos comunes y evitar duplicación.

Objetos Input

Los Inputs representan los datos que recibe un endpoint.

Por ejemplo:

```
Inputs
└── ProductCreateInput.cs
```

```
public class ProductCreateInput
{
   public string Name { get; set; }
   public decimal Price { get; set; }
}
```

El endpoint puede recibir:

POST /api/products body: {"name": "Teclado", "price": 25000 }

El Input representa específicamente **qué datos puede enviar el cliente**.

Objetos Output

Los Outputs representan la información que devolverá un endpoint.

Por ejemplo:

```
public class ProductOutput
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

Esto permite evitar que necesariamente devolvamos directamente nuestras entidades.

Podemos tener:



Entity vs Input vs Output

Este concepto es muy importante.No son necesariamente el mismo objeto.

Entity:

Representa información utilizada internamente y/o persistida, por ejemplo **Product**.

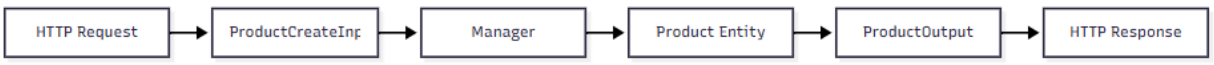
Input

Representa información que entra desde HTTP., por ejemplo **ProductCreateInput**

Output

Representa información que sale hacia el cliente, por ejemplo **ProductOutput**

Conceptualmente:



Esta separación es muy útil para evitar que la estructura interna de la aplicación quede directamente expuesta a los consumidores de la API.

Configuración de Dependency Injection

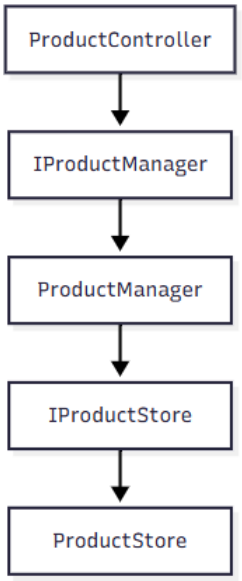
Una vez creados los componentes, debemos registrar sus implementaciones.

Por ejemplo, en la API:

builder.Services.AddScoped<IProductStore, ProductStore>();
builder.Services.AddScoped<IProductManager, ProductManager>();

De esta manera, cuando **ProductManager** solicite **IProductStore**, .NET sabrá que debe proporcionar **ProductStore**

El flujo será:



Esto permite trabajar con **interfaces y Dependency Injection**, reduciendo el acoplamiento entre las diferentes implementaciones.

Configuración de Entity Framework Core

En la API se configurará AppDbContext

Por ejemplo:

```
builder.Services.AddDbContext<AppDbContext>(options => options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

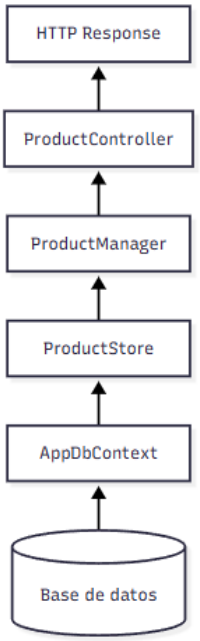
La cadena de conexión podrá estar en **appsettings.json**

Por ejemplo:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "..."
  }
}
```

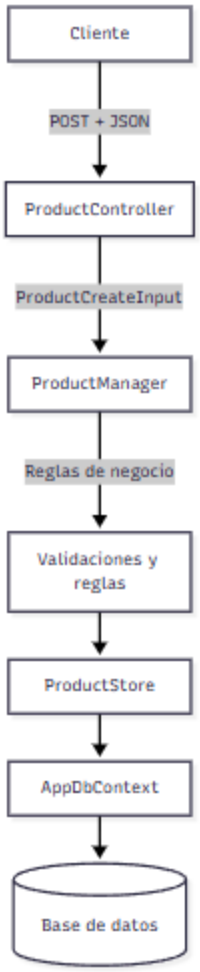
Flujo completo de una operación

Supongamos que el cliente solicita GET /api/products/10. El recorrido será:

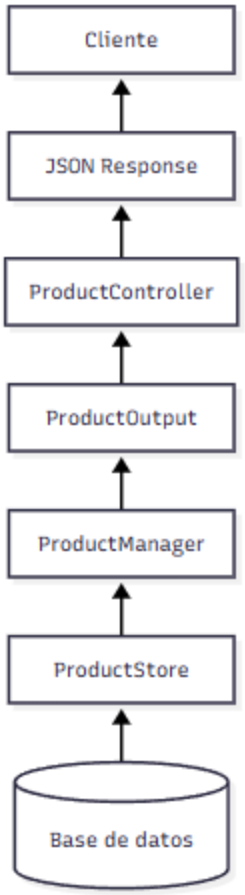


Ejemplo completo de creación

Para una operación POST /api/products el flujo puede ser:


Cliente
POST + JSON
ProductController
ProductCreateInput
ProductManager
Reglas de negocio
Validaciones y reglas
ProductStore
AppDbContext
Base de datos

Luego:



Estructura final recomendada

Una solución real podría terminar teniendo una estructura similar a:

```
MiApi
├── backend
│   └── MiApi.Api                # Host / Composition Root
│       ├── Program.cs
│       ├── appsettings.json
│       └── Properties/
│
├── core
│   └── MiApi.Core               # Transversal / Compartido
│       ├── Constants
│       ├── Helpers
│       └── Utils
│
```

```
└── layers
    ├── MiApi.Presentation          # Capa de Entrada / Exposición
    │   ├── Base
    │   ├── Controllers
    │   ├── Inputs                  # DTOs / Request models
    │   └── Outputs                 # DTOs / Response models
    │
    ├── MiApi.BusinessLogic         # Reglas de Negocio
    │   ├── Interfaces
    │   ├── Managers
    │   └── Services
    │
    ├── MiApi.DataAccess            # Persistencia
    │   ├── Interfaces
    │   ├── Stores
    │   └── AppDbContext.cs
    │
    └── MiApi.Entities              # Dominio Puro
        ├── Entities
        └── Enums
```